

1. What If There Were No Operating System?

Without an operating system, a user would find it very difficult to use a computer because they would have to directly communicate with hardware.

Problems Without an Operating System

Without an OS:

1. No easy way to run programs.
2. No proper file management.
3. No easy communication with hardware devices.
4. No memory management.
5. No multitasking.
6. No user interface.
7. No security or user access control.

Without OS vs With OS

Feature	Without Operating System	With Operating System
File Management	User cannot easily create or organize files	OS manages files and folders
Program Execution	Difficult to run applications	OS loads and executes programs
Memory Management	Programs must manage memory directly	OS allocates and manages memory
Hardware Management	User must communicate directly with hardware	OS controls keyboard, mouse, printer, disk, etc.
Multitasking	Difficult to run multiple programs	Multiple programs can run together
Security	No user authentication or permissions	OS provides users, passwords, and permissions
User Interface	No simple interface	CLI or GUI is available

Linux Demonstration of OS Features

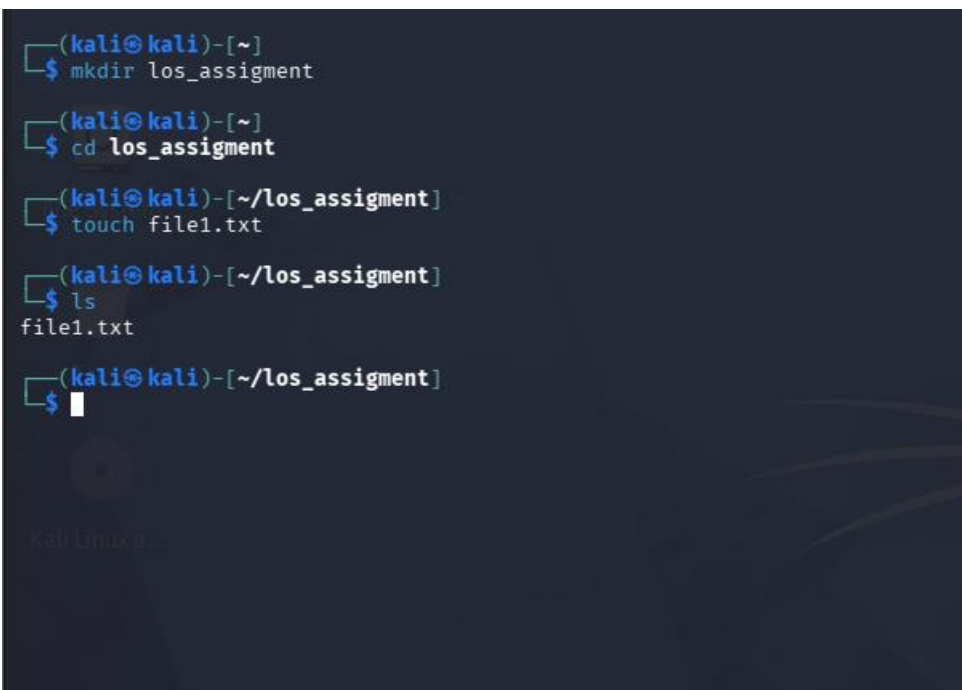
1:File Management

File management is an operating system feature that helps users create, organize, store, access, and delete files and folders. In Linux, commands such as mkdir, touch, ls, and rm are used for file management.

mkdir OS_Assignment → Creates a directory.

cd OS_Assignment → Enters the directory.

touch file1.txt → Creates a file.

A terminal window screenshot from Kali Linux. The prompt is (kali@kali)-[~]. The user enters 'mkdir los_assignment'. The prompt changes to (kali@kali)-[~/los_assignment]. The user enters 'touch file1.txt'. The prompt changes to (kali@kali)-[~/los_assignment]. The user enters 'ls', and the output 'file1.txt' is displayed. The user then enters a dollar sign '\$' followed by a cursor, indicating the next command.

```
(kali@kali)-[~]  
$ mkdir los_assignment  
  
(kali@kali)-[~]  
$ cd los_assignment  
  
(kali@kali)-[~/los_assignment]  
$ touch file1.txt  
  
(kali@kali)-[~/los_assignment]  
$ ls  
file1.txt  
  
(kali@kali)-[~/los_assignment]  
$
```

2: Process Management

Process management allows the operating system to create, run, monitor, and terminate programs. Each running program is treated as a process and is assigned a unique Process ID (PID). Commands such as `ps` and `kill` can be used to view and manage processes.

`sleep` → A command that waits for a specified time.

`60` → The process will run for 60 seconds.

`&` → Runs the process in the background.

`ps` stands for Process Status.

It displays the processes currently running in the terminal.

You can see information such as PID, terminal, running time, and command name.

`kill` → Used to terminate or stop a running process.

`2456` → The PID of the process you want to stop.

```
(kali@kali)-[~/los_assignment]
$ sleep 60 &
[2] 7163

(kali@kali)-[~/los_assignment]
$ ps
  PID TTY          TIME CMD
 2444 pts/0        00:00:03 zsh
 6504 pts/0        00:00:00 sleep
 7163 pts/0        00:00:00 sleep
 7224 pts/0        00:00:00 ps

(kali@kali)-[~/los_assignment]
$ kill 7163

[2] + terminated sleep 60
(kali@kali)-[~/los_assignment]
$ ps
  PID TTY          TIME CMD
 2444 pts/0        00:00:03 zsh
 6504 pts/0        00:00:00 sleep
 7380 pts/0        00:00:00 ps
```

3: User and Security Management

User and security management helps the operating system identify users and protect files and system resources. Linux uses user accounts, passwords, groups, and file permissions to control access. Commands such as `whoami`, `id`, and `ls -l` can be used to view user and permission information.

`whoami` → Shows the current user.

`id` → Shows user ID and group information.

```
(kali@kali) ~/los_assignment
$ whoami
kali
(kali@kali) ~/los_assignment
$ id
uid=1000(kali) gid=1000(kali) groups=1000(kali),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),100(users),101(netdev),102(scanner),114(wireshark),119(kaboxer),979(blueoath),999(lpadmin)
(kali@kali) ~/los_assignment
$
```

2. Cybersecurity Laboratory – Linux OS Selection Guide

Comparison Chart

Linux Distribution	Main Purpose	Suitable Role	Important Features
Kali Linux	Penetration Testing	Ethical Hacker / Pen Tester	Security testing tools
Parrot OS	Security and Privacy	Security Researcher	Security tools and privacy features
Ubuntu	General Linux and Server Security	System Administrator	Stable and beginner-friendly
CAINE	Digital Forensics	Digital Forensic Investigator	Forensics and investigation tools
Security Onion	Security Monitoring	SOC Analyst	Network monitoring and threat detection
Tails	Privacy and Anonymity	Privacy Researcher	Privacy-focused environment

OS Selection Guide

1. Penetration Testing → Kali Linux

Recommended for: Ethical hackers and penetration testers.

Features:

- Nmap
- Wireshark
- Burp Suite
- Metasploit Framework
- Many security testing tools

Example use:

A penetration tester can use Kali Linux to scan and test systems in an authorized lab environment.

2. Privacy and Security → Parrot OS

Recommended for: Security researchers and privacy-focused users.

Features:

- Security tools
 - Privacy tools
 - Lightweight environment
 - Suitable for penetration testing and development
-

3. Digital Forensics → CAINE Linux

Recommended for: Digital forensic investigators.

Features:

- Data recovery tools
- Disk analysis tools
- Digital investigation environment
- Evidence analysis support

Example use:

An investigator can analyze a disk image and search for deleted files.

4. Security Monitoring → Security Onion

Recommended for: SOC analysts and security monitoring teams.

Features:

- Network security monitoring
- Intrusion detection
- Log analysis
- Threat detection

Example use:

A SOC analyst can monitor network traffic and investigate suspicious activity.

5. Privacy and Anonymity → Tails

Recommended for: Privacy-focused users.

Features:

- Privacy-focused design
- Leaves minimal traces on the computer when used as intended
- Designed for anonymous communication

3. Exit Status in Linux

After executing a command, Linux returns an exit status.

The exit status tells us whether the command was successful or unsuccessful

```
(kali㉿kali)-[~/los_assignment]
$ pwd
/home/kali/los_assignment

(kali㉿kali)-[~/los_assignment]
$ echo $?
0

(kali㉿kali)-[~/los_assignment]
$ ls folder
ls: cannot access 'folder': No such file or directory

(kali㉿kali)-[~/los_assignment]
$ echo $?
2

(kali㉿kali)-[~/los_assignment]
$ touch file2.txt

(kali㉿kali)-[~/los_assignment]
$ echo $?
0

(kali㉿kali)-[~/los_assignment]
$ mkdir testfolder

(kali㉿kali)-[~/los_assignment]
$ echo $?
0

(kali㉿kali)-[~/los_assignment]
$ echo "Hello"
Hello

(kali㉿kali)-[~/los_assignment]
$ echo $?
0

(kali㉿kali)-[~/los_assignment]
$ echo $(wrongfolder)
wrongfolder: command not found

(kali㉿kali)-[~/los_assignment]
$ echo $?
0
```


Command	Result	Exit Status	Meaning
pwd	Successfully displayed /home/kali/os_assignment	0	Command executed successfully
ls folder	Failed – folder does not exist	2	Error: No such file or directory
touch test.txt	Successfully created test.txt	0	Command executed successfully
mkdir testfolder	Successfully created testfolder	0	Command executed successfully
echo "Hello"	Successfully displayed Hello	0	Command executed successfully

Script Learn From an Exit Status

```
mkdir demo
```

```
if [ $? -eq 0 ]
```

```
then
```

```
    echo "Folder created successfully"
```

```
else
```

```
    echo "Folder creation failed"
```

```
fi
```

```
(kali@kali)~[/los_assignment]
$ touch exit_status.sh
(kali@kali)~[/los_assignment]
$ gedit exit_status.sh
** (gedit:13653): WARNING **: 20:44:17.634: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found
** (gedit:13653): WARNING **: 20:44:17.634: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found
(kali@kali)~[/los_assignment]
$ chmod 777 exit_status.sh
(kali@kali)~[/los_assignment]
$ ./exit_status.sh
Folder created successfully
(kali@kali)~[/los_assignment]
$ ./exit_status.sh
mkdir: cannot create directory 'demo': File exists
Folder creation failed
(kali@kali)~[/los_assignment]
$ echo $?
0
(kali@kali)~[/los_assignment]
$
```